

Anexo tema 2: Funciones en Java

Índice

1. Sintaxis de una función en Java
2. Sobrecarga de funciones en Java
3. Documentación de clases (JavaDoc)
4. Ejemplo: suma de dos valores de tipo entero o decimal
5. Recursividad

Las funciones en Java, también llamadas métodos, nos permiten dividir nuestro código en bloques modulares y reutilizables, facilitando la organización, el mantenimiento y la legibilidad de nuestros programas

Una función es un bloque de código que realiza una tarea específica y se ejecuta cuando es llamado. Las funciones pueden recibir datos como entrada (parámetros), procesar estos datos y devolver un resultado (valor de retorno).

El uso de funciones nos permite reutilizar y organizar nuestro código de manera más eficiente, ya que podemos dividir un programa en partes más pequeñas y modulares.

En Java, las funciones se definen dentro de una clase y se utilizan para manipular el estado (atributos) y comportamiento (métodos) de los objetos de esa clase.

1. Sintaxis de una función en Java

```
modificadorAcceso tipoRetorno nombreFuncion (tipoParametro1
nombreParametro1, tipoParametro2 nombreParametro2, ...) {
    // Cuerpo de la función
    // Procesamiento y cálculos
    return valorRetorno;
}
```

modificadorAcceso: Controla la visibilidad de la función en relación con otras clases y objetos. Puede ser public, private, protected o sin modificador (por defecto).

- **public:** La función es accesible desde cualquier clase.
- **private:** La función sólo es accesible dentro de la misma clase en la que se define.
- **protected:** La función es accesible dentro de la misma clase y sus subclases.

- **Sin modificador** (por defecto): La función es accesible dentro del mismo paquete.

tipoRetorno: Indica el tipo de dato que la función devolverá. Puede ser un tipo primitivo (int, float, etc.), una clase, una interfaz o void si la función no devuelve ningún valor.

nombreFuncion: Es el nombre único que identifica a la función y sigue las convenciones de camelCase.

tipoParametro y nombreParametro: Son los parámetros que recibe la función como entrada. Pueden ser de cualquier tipo de dato, y su cantidad puede variar. Si la función no recibe parámetros, se dejan los paréntesis vacíos.

return valorRetorno; Es la instrucción para devolver un valor desde la función. Si la función tiene un tipo de retorno void, no se utiliza la declaración return.

2. Sobrecarga de funciones en Java

La sobrecarga de funciones permite definir varias funciones con el mismo nombre. La forma que tiene de saber qué función es la que tiene que ejecutar es mediante el número y tipo de los parámetros que se le pasan, es decir, cada función tendrá un número único de parámetros y tipos para poder identificarse de otra función.

3. Documentación de clases (JavaDoc)

Hasta este momento hemos usado los comentarios para describir partes o bloques de nuestro programa. Java tiene una utilidad que realiza la documentación en formato HTML que automatiza esta tarea. Esta utilidad es el que se utiliza generalmente para la documentación de programas y se llama JavaDoc.

Un comentario JavaDoc comienza por `/**` (dos asteriscos) y termina con `*/` (como los comentarios multilínea normales).

Dentro de ellos cada línea puede comenzar por espacios, seguidos de `*` y del contenido real. Ejemplo:

```
/**
 * Este es un comentario JavaDoc
 */
```

Los asteriscos al inicio de la línea no forman parte de la documentación.

Dentro de un comentario JavaDoc se puede incluir HTML pero nosotros no vamos a emplearlo excepto para alguna negrita o cursiva.

Además del texto, se puede emplear etiquetas (tags) que proporcionan información, la cual se recopila y con las que se hacen tablas o referencias cruzadas. Las etiquetas comienzan por el carácter arroba (@). Entre otras tenemos las siguientes etiquetas:

Tag	Descripción	Uso
@author	Nombre del desarrollador.	nombre_autor
@version	Versión del método o clase.	versión
@param	Definición de un parámetro de un método, es requerido para todos los parámetros del método.	nombre_parametro
@return	Informa de lo que devuelve el método, no se puede usar en constructores o métodos "void".	descripción
@throws	Excepción lanzada por el método, posee un sinónimo de nombre @exception nombre_clase	descripción

El comentario se coloca justo antes de lo que se va a comentar: clase, atributo, método.

Para los atributos, ya que normalmente son privados basta con un comentario "normal", pero la clase y los métodos si deberían ir comentados, incluyendo los privados o protegidos.

```
/**
 * Clase que representa a una Persona.
 * Una persona es alguien que participa en el sistema, ya sea como
 * empleado o como cliente
 * @author Francis Fernández
 * @version 1.0
 */
```

Ejemplo de documentación de clase

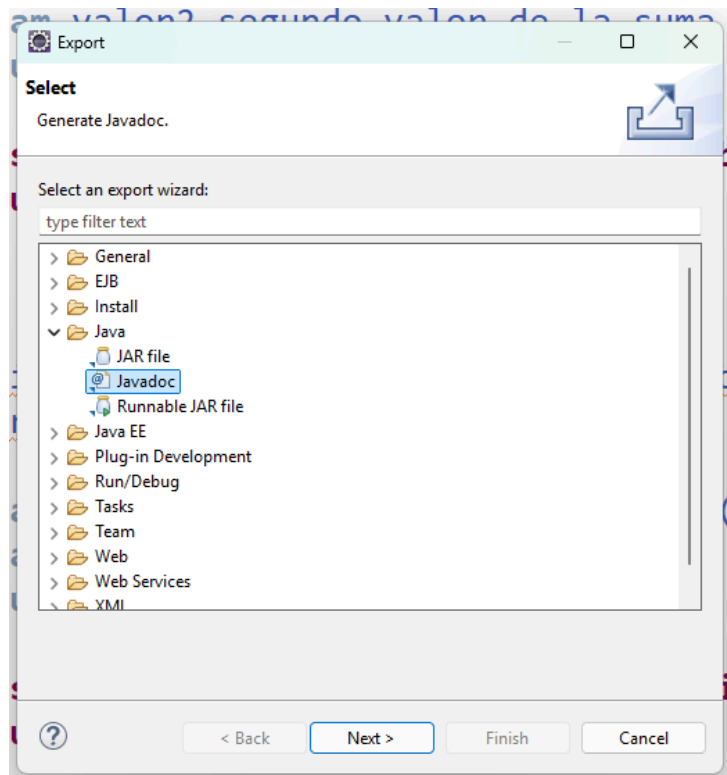
```
public class Persona {  
  
}
```

Para los métodos podría ser algo como lo siguiente:

```
/**  
 * Calcula la media de tres valores  
 * @param valor1 Primer valor  
 * @param valor2 Segundo valor  
 * @param valor3 Tercer valor  
 * @return Media de los tres valores  
 * @author Francis Fernández  
 */  
public double calculaMedia3(double valor1, double valor2, double  
    valor3) {  
    return (valor1 + valor2 + valor3) / 3.0;  
}
```

Una vez que tenemos todas nuestras clases comentadas, la utilidad javadoc, desde la línea de comandos nos generará automáticamente archivos HTML navegables con la documentación de nuestras clases.

Si usamos Eclipse, este también puede hacerlo de forma muy sencilla y sin usar la línea de comandos. (File>Export>Java>Javadoc).



4. Ejemplo: suma de dos valores de tipo entero o decimal

```
package calculadora;

import java.util.Scanner;

/**
 * Programa que calcula la suma de dos números tipo entero o double, con
 * cualquier combinación entre ellos. Hace uso de varias elementos de
 * programación no vistos hasta ahora, como son la modularidad en Java,
 * mediante la creación de funciones, sobrecarga de funciones (o
 * polimorfismo), en la que podemos declarar una función varias veces con el
 * mismo nombre y dependiendo del tipo de los argumentos se ejecutará una u
 * otra.
 *
 * @author Francis Fernández
 * @version 1.0 10/10/2025
 */
public class Calculadora {

    /**
     * Módulo principal
     */
}
```

```

* @param args no usado
*/
public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);
    System.out.println("SUMA DE DOS VALORES. EJEMPLO DE CLASE\n"
        + "Introduzca la opción:\n"
        + "1: Suma de decimales\n"
        + "2: Suma de enteros\n"
        + "3: Suma de un entero y un decimal\n"
        + "4: Suma de un decimal y un entero\n");

    int opcion = Integer.parseInt(sc.nextLine());
    double resultado;

    switch (opcion) {
    case 1:
        suma();
        break;
    case 2: {
        System.out.print("Introduzca el primer valor (decimal): ");
        double valor1 = Double.parseDouble(sc.nextLine());
        System.out.print("Introduzca el segundo valor (entero): ");
        int valor2 = Integer.parseInt(sc.nextLine());
        resultado = suma(valor1, valor2);
        System.out.println("La suma es:" + resultado);
        break;
    }

    case 3:
        System.out.print("Introduzca el primer valor (entero): ");
        int valor1 = Integer.parseInt(sc.nextLine());
        System.out.print("Introduzca el primer valor (entero): ");
        int valor2 = Integer.parseInt(sc.nextLine());
        resultado = suma(valor1, valor2);
        System.out.println("La suma es:" + resultado);
        break;

    case 4:
        System.out.print("Introduzca el primer valor (entero): ");
        int valor3 = Integer.parseInt(sc.nextLine());
        System.out.print("Introduzca el primer valor (decimal): ");
        double valor4 = Double.parseDouble(sc.nextLine());
        resultado = suma(valor3, valor4);
        System.out.println("La suma es:" + resultado);
        break;
    }
}

```

```

        default:
            System.out.println("El valor introducido no es válido");
        }
        sc.close();
    }

    /**
     * Procedimiento que suma dos valores tipo Double
     *
     */
    public static void suma() {
        Scanner sc = new Scanner(System.in);
        System.out.print("Introduzca el primer valor (decimal): ");
        double valor1 = Double.parseDouble(sc.nextLine());
        System.out.print("Introduzca el segundo valor (decimal): ");
        double valor2 = Double.parseDouble(sc.nextLine());

        System.out.println("La suma es:" + (valor1 + valor2));
        sc.close();
    }

    /**
     * Función que suma dos valores de tipo entero
     *
     * @param valor1 primer valor de la suma (int)
     * @param valor2 segundo valor de la suma (int)
     * @return suma de los dos valores
     */
    public static int suma(int valor1, int valor2) {
        return valor1 + valor2;
    }

    /**
     * Función que suma dos valores el primero de tipo entero y el segundo de tipo
     * double
     *
     * @param valor1 primer valor de la suma (int)
     * @param valor2 segundo valor de la suma (double)
     * @return suma de los dos valores
     */
    public static int suma(int valor1, double valor2) {
        return valor1 + (int) valor2;
    }

    /**
     * Función que suma dos valores el primero de tipo double y el segundo de tipo
     * entero

```

```

*
* @param valor1 primer valor de la suma (double)
* @param valor2 segundo valor de la suma (int)
* @return suma de los dos valores
*/
public static int suma(double valor1, int valor2) {
    return (int) valor1 + valor2;
}
}

/*
* Como ejercicio se propone que el alumno modifique el código para que este
* programa para que se repita hasta que el usuario desee. Para ello, habrá que
* introducir la opción número 5, donde se podrá salir del programa.
* Además, elimina los errores que pudiera haber en el código
*/

```

5. Recursividad

Una función recursiva es una función que se llama a sí misma durante su ejecución. La recursión es una técnica poderosa en la programación que puede simplificar la solución de ciertos problemas, como el cálculo de factoriales, la secuencia de Fibonacci y el recorrido de estructuras de datos jerárquicas.

Para evitar una recursión infinita, las funciones recursivas deben tener una condición de salida o base que detenga la recursión en un punto específico.

Es importante garantizar que la función alcance esta condición base, ya que, de lo contrario, la función se llamará a sí misma indefinidamente, lo que podría agotar los recursos del sistema y provocar un error en tiempo de ejecución.

Ejemplo: Calcular el factorial de un número utilizando recursión

```

package recursividad;
/**

```



```

* Programa que realiza el factorial de un número
* @author Francis Fernández
* @version 1.1 22/10/25
*/
public class Main {
    /**
     * Método principal
     * @param args no utilizado
     */
    public static void main(String[] args) {

        long factorial5 = factorial(5); // 5! = 5 * 4 * 3 * 2 * 1 = 120
        System.out.println("Factorial de 5: " + factorial5);
        long factorial10 = factorial(20); // 10! = 3628800
        System.out.println("Factorial de 10: " + factorial10);
    }

    /**
     * Función recursiva para calcular el factorial
     * @param numero numero a realizar el factorial
     * @return resultado del factorial
     */
    public static long factorial(long numero) {
        // Condición base: si el número es 0 o 1, el factorial es 1
        if (numero <= 1) {
            return 1;
        }
        // Llamada recursiva: multiplicar el número por el factorial de (número - 1)
        return numero * factorial(numero - 1);
    }
}

```